

International Cybersecurity and Digital Forensics Academy (ICDFA)**CIP-B103: Network Forensics Fundamentals****LAB 1: HTTP Analysis Using Wireshark - Text Traffic****Three-Week Delivery Block 1/3 | 18-19 July 2026**

Prepared by: Aminu Idris, AMCPN | Founder, ICDFA

Important Pre-Lab Instruction Read the matching lecture slide deck before executing the practical. Work only inside the ICDFA-approved lab or your own isolated virtual environment. Record every command, observation and screenshot as contemporaneous evidence.	
Course Code	CIP-B103
Course Title	Network Forensics Fundamentals
Student Name	Fuseini Imoru Kantuogaa
Student Reg. Number	17291
Instructor Name	Dr. Sarah James
Lab Number	Lab 1
Lab Title	HTTP Analysis Using Wireshark - Text Traffic
Delivery Block	1/3 of 3
Scheduled Dates	18-19 July 2026
Estimated Duration	3-4 hours practical work plus report writing
Mode	Individual practical lab in an authorized virtual environment
Required Evidence	basic.pcapng or basic.log generated locally; optional supplied basic.log

1. Source Materials and Slide Alignment

- HTTP, Wireshark and Digital Forensics Part 1: HTTP request/response, OSI encapsulation, TCP three-way handshake, sequence and acknowledgement numbers, connection termination and evidence reporting.

- The exercise reproduces a simple local Apache page, captures plaintext HTTP traffic, and inspects frame, IP, TCP and HTTP metadata.
- The slide assignment requires a named webpage, sender/receiver ports, initial sequence numbers, timestamps, IP addresses, MAC addresses and HTTP request timing.

Reading Requirement Before Lab Execution

The lab deliberately follows the sequence, terminology and core exercises presented in the uploaded CIP-B103 slides. Commands have been corrected where needed for Linux syntax and forensic repeatability.

2. Lab Scenario

You are a junior network forensic analyst asked to document a short plaintext HTTP session. You will create a local webpage containing your name and registration number, capture the traffic, reconstruct the TCP conversation, identify the HTTP request and response, and explain how each protocol layer contributes evidence.

3. Learning Outcomes

- Explain the forensic value of HTTP, TCP, IP and Ethernet metadata.
- Create and serve a simple HTML page using Apache.
- Capture HTTP traffic using Wireshark or tshark without altering the evidence.
- Identify the SYN, SYN-ACK and ACK packets in a TCP handshake.
- Extract source/destination ports, sequence numbers, acknowledgement numbers, timestamps and HTTP headers.
- Explain why loopback captures do not show normal Ethernet MAC addresses and repeat the capture on a two-VM network where required.
- Generate a concise network forensic timeline and evidence report.

4. Legal, Ethical and Safety Rules

- Use only systems, packet captures and networks for which you have explicit authorization.
- Keep the lab isolated from production, campus, office and public networks. Use NAT or host-only virtual networking as instructed.
- Do not collect, disclose or reuse real credentials, personal messages or confidential traffic.
- Preserve original capture files. Perform analysis on verified working copies and record SHA-256 hashes.

- Stop any traffic-generation or interception process immediately after the required evidence is captured.
- Restore firewall, ARP, forwarding and network settings before closing the lab.
- Do not browse to credential-bearing or private HTTP sites for this exercise. Use only the local page or the instructor-approved NeverSSL training page.

5. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux or Ubuntu VM	Run Apache, curl, tshark and Wireshark.
Web server	Apache2	Serve the training webpage locally.
Capture tools	Wireshark and tshark	Capture and inspect network packets.
Client	curl and a web browser	Generate controlled HTTP requests.
Hashing	sha256sum	Verify capture integrity.
Optional second host	Windows or Linux VM	Produce Ethernet frames with visible MAC addresses.

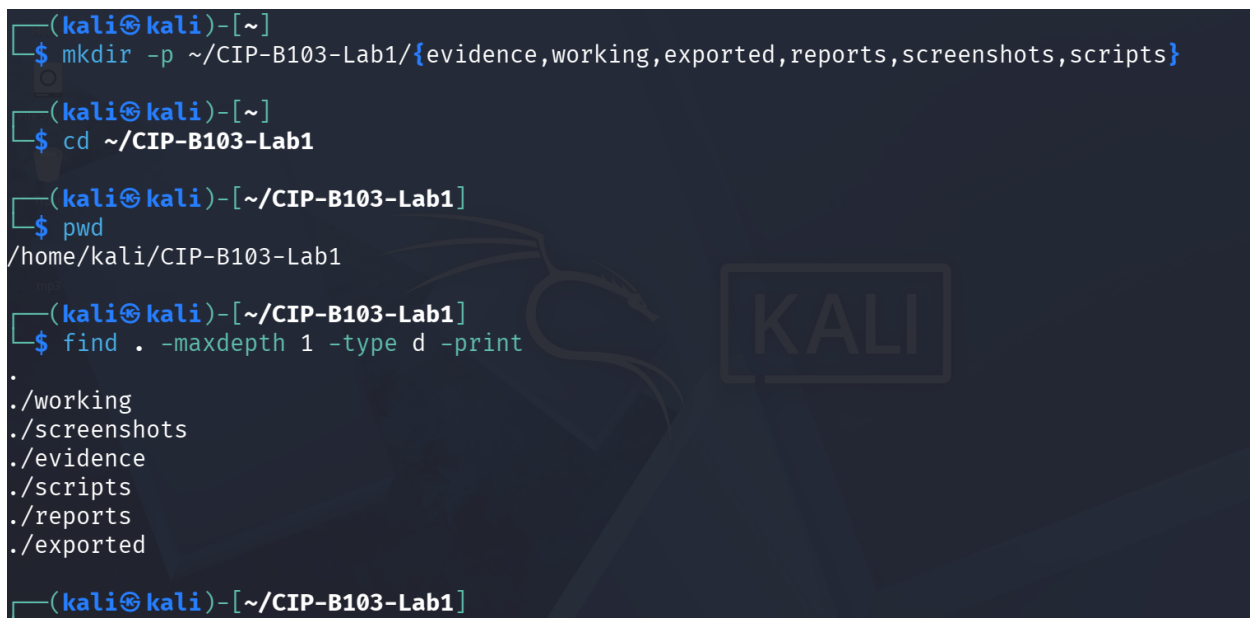
6. Lab Folder Structure and Evidence Preparation

```
mkdir -p ~/CIP-B103-Lab1/{evidence,working,exported,reports,screenshots,scripts}  
cd ~/CIP-B103-Lab1  
pwd  
find . -maxdepth 1 -type d -print
```

Create the folder structure before downloading or generating evidence. Store original captures under evidence and analysis copies under working.

```
sudo apt update
sudo apt install -y apache2 curl wireshark tshark
sudo systemctl enable --now apache2
printf '<!DOCTYPE html>\n<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: YOUR
FULL NAME</p><p>Reg No: YOUR REGISTRATION NUMBER</p></body></html>\n' | sudo tee
/var/www/html/basic.html
curl -v http://127.0.0.1/basic.html
# After capture:
sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt
```

Screenshot required: Apache status, the completed webpage in a browser, folder structure and the SHA-256 of the capture.

A terminal window screenshot from a Kali Linux system. The prompt is (kali㉿kali)-[~]. The user enters \$ mkdir -p ~/CIP-B103-Lab1/{evidence,working,exported,reports,screenshots,scripts}. The prompt changes to (kali㉿kali)-[~/CIP-B103-Lab1]. The user enters \$ cd ~/CIP-B103-Lab1. The prompt changes to (kali㉿kali)-[~/CIP-B103-Lab1]. The user enters \$ pwd. The output is /home/kali/CIP-B103-Lab1. The user enters \$ find . -maxdepth 1 -type d -print. The output lists the following directories: ./working, ./screenshots, ./evidence, ./scripts, ./reports, and ./exported. The prompt changes to (kali㉿kali)-[~/CIP-B103-Lab1].

```
(kali㉿kali)-[~]
$ mkdir -p ~/CIP-B103-Lab1/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd ~/CIP-B103-Lab1

(kali㉿kali)-[~/CIP-B103-Lab1]
$ pwd
/home/kali/CIP-B103-Lab1

(kali㉿kali)-[~/CIP-B103-Lab1]
$ find . -maxdepth 1 -type d -print
.
./working
./screenshots
./evidence
./scripts
./reports
./exported

(kali㉿kali)-[~/CIP-B103-Lab1]
```

Screenshot 1: Evidence folder structure created (mkdir, cd, pwd, find).

```
(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease
Get:2 http://kali.download/kali kali-rolling/main amd64
Get:3 http://kali.download/kali kali-rolling/main amd64
Get:4 http://kali.download/kali kali-rolling/contrib am
Get:5 http://kali.download/kali kali-rolling/contrib am
Fetched 74.7 MB in 4min 36s (270 kB/s)
1736 packages can be upgraded. Run 'apt list --upgradab

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo apt install -y apache2 curl wireshark tshark
apache2 is already the newest version (2.4.68-1).
curl is already the newest version (8.20.0-5).
wireshark is already the newest version (4.6.6-1).
tshark is already the newest version (4.6.6-1).
The following packages were automatically installed and
  libraw23t64 libvidstab1.1 openjdk-21-jre openjdk-2
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgradi
```

Screenshot 2: Package list updated and apache2, curl, wireshark, tshark installed.

```
(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo systemctl enable --now apache2
Synchronizing state of apache2.service with SysV servic
install.
Executing: /usr/lib/systemd/systemd-sysv-install enable

(kali㉿kali)-[~/CIP-B103-Lab1]
$ systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.se
   Active: active (running) since Tue 2026-07-28 07:3
   Invocation: dce5043be1b5485aabe10a1bbf6f1305
     Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 966 (apache2)
   Status: "Total requests: 0; Idle/Busy workers 100/
   Tasks: 6 (limit: 2075)
   Memory: 21.1M (peak: 21.9M)
     CPU: 348ms
   CGroup: /system.slice/apache2.service
           └─ 966 /usr/sbin/apache2 -k start -DFOREGR
              1008 /usr/sbin/apache2 -k start -DFOREGR
              1009 /usr/sbin/apache2 -k start -DFOREGR
              1010 /usr/sbin/apache2 -k start -DFOREGR
```

Screenshot 3: Apache2 enabled and systemctl status showing the service starting.

```
(kali㉿kali)-[~/CIP-B103-Lab1]
$ printf '<!DOCTYPE html>\n<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fuseini Imoru</p><p>Reg No: 17291</p></body></html>\n' | sudo tee /var/www/html/basic.html
<!DOCTYPE html>
<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fuseini Imoru</p><p>Reg No: 17291</p></body></html>

(kali㉿kali)-[~/CIP-B103-Lab1]
$ curl -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 38024
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 28 Jul 2026 11:50:40 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 28 Jul 2026 11:50:25 GMT
< ETag: "78-657aa71cb8292"
```

Screenshot 4: basic.html created with printf and the initial curl -v request/response headers.

```
< Last-Modified: Tue, 28 Jul 2026 11:50:25 GMT
< ETag: "78-657aa71cb8292"
< Accept-Ranges: bytes
< Content-Length: 120
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fuseini Imoru</p><p>Reg No: 17291</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact
```

Screenshot 5: Continued curl -v output showing the HTML body and closed connection.

7. Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	CIP-B103-Lab1-YourFullName
Trainee name	Fuseini Imoru
Date and time started	Tue 28 July 2026, approx. 07:35 EDT (Kali VM) — inferred from the Apache2 service start time (07:35:31 EDT) shown in systemctl status
Evidence file name(s)	basic.pcapng (working copy: basic_working.pcapng)
Source or generation method	Captured locally with tshark on loopback (lo, tcp port 80) while curl requested http://127.0.0.1/basic.html served by the local Apache2 instance

Field	Student Entry
Original SHA-256	53d0cadf491d24b130821c786a5d4a9478edd0a01986aeae0d1b36b2a647334f (preliminary Part A test capture, superseded) — final evidence/basic.pcapng used for analysis: 2e8b38d4cddf38b0e338f689287b45ebfc4ff939731da83e29fbfe5ea162617e
Working-copy SHA-256	2e8b38d4cddf38b0e338f689287b45ebfc4ff939731da83e29fbfe5ea162617e (working/basic_working.pcapng) — identical to the final evidence copy
Analysis workstation/VM	Kali Linux (user: kali), CIP-B103-Lab1 working directory
Notes on any changes	A first, informal capture/hash pair was made while verifying the webpage in Part A and was superseded by the official Part B capture. cp -- preserve=timestamps was used for the working copy; SHA-256 of the final evidence and working copies matched exactly, confirming no alteration.

8. Part A - Verify the Web Service and Generate Text HTTP Traffic

1. Replace the name and registration number placeholders in basic.html with your own details.
2. Open `http://127.0.0.1/basic.html` and confirm that the page displays correctly.
3. Run curl with verbose output and save the request/response headers.
4. Record the local service state and listening port.

```
sudo systemctl status apache2 --no-pager
```

```
sudo ss -lntp | grep ':80'
```

```
curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
```

```

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo tshark -i lo -w /tmp/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10 ^C

(kali㉿kali)-[~/CIP-B103-Lab1]
$ cp /tmp/basic.pcapng ~/CIP-B103-Lab1/evidence/
cp: cannot open '/tmp/basic.pcapng' for reading: Permission denied

(kali㉿kali)-[~/CIP-B103-Lab1]
$ ls -l /tmp/basic.pcapng
-rw----- 1 root root 1812 Jul 28 07:56 /tmp/basic.pcapng

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo cp /tmp/basic.pcapng ~/CIP-B103-Lab1/evidence/

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo chown kali:kali ~/CIP-B103-Lab1/evidence/basic.pcapng

```

Screenshot 6: First loopback tshark capture attempt, corrected for a permission error and chown'd.

```

zsh: corrupt history file /home/kali/.zsh_history
(kali㉿kali)-[~/CIP-B103-Lab1]
$ curl -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 39834
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 28 Jul 2026 11:55:40 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 28 Jul 2026 11:50:25 GMT
< ETag: "78-657aa71cb8292"
< Accept-Ranges: bytes
< Content-Length: 120
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fus
eini Imoru</p><p>Reg No: 17291</p></body></html>

```

Screenshot 7: Repeat curl -v request to /basic.html generating a second HTTP exchange.


```

(kali㉿kali)-[~/CIP-B103-Lab1]
$ ls -l ~/CIP-B103-Lab1/evidence/basic.pcapng
-rw-r--r-- 1 kali kali 1812 Jul 28 07:57 /home/kali/CIP-B103-Lab1/evidence/basic.pcapng

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sha256sum evidence/basic.pcapng | tee reports/basic_pcap_sha256.txt
53d0cadf491d24b130821c786a5d4a9478edd0a01986aeae0d1b36b2a647334f evidence/basic.pcapng

(kali㉿kali)-[~/CIP-B103-Lab1]
$ tshark -r evidence/basic.pcapng -Y http
  4 0.000168092      127.0.0.1 → 127.0.0.1      HTTP 149
GET /basic.html HTTP/1.1
  6 0.001945901      127.0.0.1 → 127.0.0.1      HTTP 437
HTTP/1.1 200 OK (text/html)

(kali㉿kali)-[~/CIP-B103-Lab1]
$

```

Screenshot 8: Evidence file listed, SHA-256 hashed, and inspected with tshark -Y http.

```

(kali㉿kali)-[~/CIP-B103-Lab1]
$ curl -v http://127.0.0.1/basic.html 2>&1 | tee reports/curl_verbose.txt
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 38292
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0      0     0  0* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 28 Jul 2026 12:10:18 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 28 Jul 2026 11:50:25 GMT
< ETag: "78-657aa71cb8292"
< Accept-Ranges: bytes
< Content-Length: 120
< Vary: Accept-Encoding
< Content-Type: text/html

```

Screenshot 9: Part A curl -v request piped to reports/curl_verbose.txt.

```

<
{ [120 bytes data]
100 120 100 120 0 0 62337 0
* Connection #0 to host 127.0.0.1:80 left intact
<!DOCTYPE html>
<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fuseini Imoru</p><p>Reg No: 17291</p></body></
tml>

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo systemctl status apache2 --no-pager
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-07-28 07:35:31 EDT; 35min ago
  Invocation: dce5043be1b5485aabe10a1bbf6f1305
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 966 (apache2)
   Status: "Total requests: 3; Idle/Busy workers 100/0;Requests/sec: 0.00143; Bytes served/sec:
0 B/sec"
      Tasks: 6 (limit: 2075)
     Memory: 16.7M (peak: 21.9M, swap: 2.8M, swap peak: 2.8M)
        CPU: 501ms
    CGroup: /system.slice/apache2.service
           └─ 966 /usr/sbin/apache2 -k start -DFOREGROUND

```

Screenshot 10: curl_verbose.txt continued and systemctl status apache2 --no-pager.

```

└─1008 /usr/sbin/apache2 -k start -DFOREGROUND
└─1009 /usr/sbin/apache2 -k start -DFOREGROUND
└─1010 /usr/sbin/apache2 -k start -DFOREGROUND
└─1011 /usr/sbin/apache2 -k start -DFOREGROUND
└─1012 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 28 07:35:31 kali systemd[1]: Starting apache2.service - The Apache HTTP Server ...
Jul 28 07:35:31 kali apachectl[966]: AH00558: apache2: Could not reliably determine the serv...essage
Jul 28 07:35:31 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.

(kali㉿kali)-[~/CIP-B103-Lab1]
$ sudo ss -lntp | grep ':80'
LISTEN 0      511          *:*          users:((("apache2",pid=1012,fd=4),("apach
e2",pid=1011,fd=4),("apache2",pid=1010,fd=4),("apache2",pid=1009,fd=4),("
apache2",pid=966,fd=4))

```

Screenshot 11: systemctl status continued and ss -lntp confirming Apache listening on TCP port 80.

```

(kali㉿kali)-[~/CIP-B103-Lab1]
$ tree ~/CIP-B103-Lab1
/home/kali/CIP-B103-Lab1
├── evidence
│   ├── basic.pcapng
│   └── root_test
├── exported
├── reports
│   ├── basic_pcap_sha256.txt
│   └── curl_verbose.txt
├── screenshots
├── scripts
└── working

7 directories, 4 files

(kali㉿kali)-[~/CIP-B103-Lab1]
$

```

Screenshot 12: Evidence folder tree after Part A.

9. Part B - Capture the Session

Use either Wireshark or tshark. The loopback interface is normally lo. Start the capture before running curl and stop it after the page is received.

Terminal 1

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
```

Terminal 2

```
curl --no-keepalive -v http://127.0.0.1/basic.html
```

Stop Terminal 1 with Ctrl+C, then preserve a working copy

```
cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng
```

```
sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
```

Wireshark Alternative

Open Wireshark, select lo, apply capture filter tcp port 80, start capture, run curl, stop capture, and save as evidence/basic.pcapng.

```
(kali㉿kali)-[~/CIP-B103-Lab1]
└─$ sudo tshark -i lo -f 'tcp port 80' -w /tmp/basic.pcapng
Running as user "root" and group "root". This could be
Capturing on 'Loopback: lo'
10 ^C

(kali㉿kali)-[~/CIP-B103-Lab1]
└─$ sudo cp /tmp/basic.pcapng evidence/basic.pcapng
sudo chown kali:kali evidence/basic.pcapng

(kali㉿kali)-[~/CIP-B103-Lab1]
└─$ ls -l evidence/basic.pcapng
-rw-r--r-- 1 kali kali 1828 Jul 28 08:23 evidence/basic.pcapng

(kali㉿kali)-[~/CIP-B103-Lab1]
└─$ cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng
```

Screenshot 13: Official Part B loopback capture started, copied to evidence/, chown'd, and copied to a working copy.

```
(kali㉿kali)-[~/CIP-B103-Lab1]
$ curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80 ...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 40264
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 28 Jul 2026 12:23:06 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 28 Jul 2026 11:50:25 GMT
< ETag: "78-657aa71cb8292"
< Accept-Ranges: bytes
< Content-Length: 120
< Vary: Accept-Encoding
< Content-Type: text/html
```

Screenshot 14: curl --no-keepalive -v request generating the analyzed HTTP session.

```
<!DOCTYPE html>
<html><body><h1>CIP-B103 HTTP Evidence</h1><p>Name: Fus
eini Imoru</p><p>Reg No: 17291</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(kali㉿kali)-[~/CIP-B103-Lab1]
$
```

Screenshot 15: Continued curl output showing the HTML body and connection closure.


```
(kali@kali)-[~/CIP-B103-Lab1]
$ sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
2e8b38d4cddf38b0e338f689287b45ebfc4ff939731da83e29fbfe5
ea162617e  evidence/basic.pcapng
2e8b38d4cddf38b0e338f689287b45ebfc4ff939731da83e29fbfe5
ea162617e  working/basic_working.pcapng
```

```
(kali@kali)-[~/CIP-B103-Lab1]
$
```

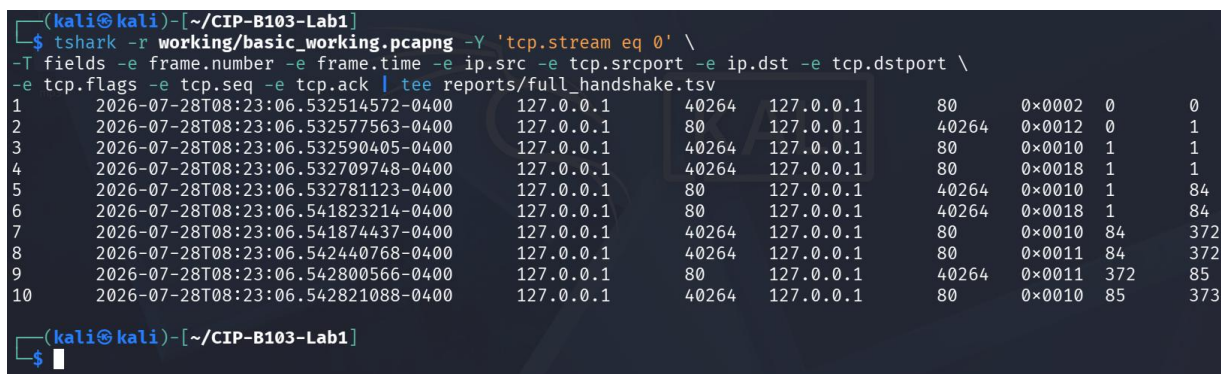
Screenshot 16: SHA-256 hashes of the evidence and working-copy captures (matching).

10. Part C - Analyze the TCP Three-Way Handshake

Open the working copy and apply the display filter `tcp.stream eq 0`. Identify the first three packets and complete the table.

Packet	Flags	Client Seq	Server Seq/ACK	Timestamp	Interpretation
1	SYN	Seq = 0 (relative ISN)	— (not yet set)	08:23:06.532515 EDT, 28 Jul 2026 (frame 1)	Client requests a TCP session.
2	SYN, ACK	—	Seq = 0, Ack = 1 (relative)	08:23:06.532578 EDT, 28 Jul 2026 (frame 2)	Server acknowledges and provides its initial sequence number.
3	ACK	Seq = 1, Ack = 1 (relative)	—	08:23:06.532590 EDT, 28 Jul 2026 (frame 3)	Client acknowledges; the connection becomes established.

```
tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.tsv
```



```
(kali@kali)~/CIP-B103-Lab1
$ tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/full_handshake.tsv
1      2026-07-28T08:23:06.532514572-0400      127.0.0.1      40264      127.0.0.1      80      0x0002      0      0
2      2026-07-28T08:23:06.532577563-0400      127.0.0.1      80      127.0.0.1      40264      0x0012      0      1
3      2026-07-28T08:23:06.532590405-0400      127.0.0.1      40264      127.0.0.1      80      0x0010      1      1
4      2026-07-28T08:23:06.532709748-0400      127.0.0.1      40264      127.0.0.1      80      0x0018      1      1
5      2026-07-28T08:23:06.532781123-0400      127.0.0.1      80      127.0.0.1      40264      0x0010      1      84
6      2026-07-28T08:23:06.541823214-0400      127.0.0.1      80      127.0.0.1      40264      0x0018      1      84
7      2026-07-28T08:23:06.541874437-0400      127.0.0.1      40264      127.0.0.1      80      0x0010      84      372
8      2026-07-28T08:23:06.542440768-0400      127.0.0.1      40264      127.0.0.1      80      0x0011      84      372
9      2026-07-28T08:23:06.542800566-0400      127.0.0.1      80      127.0.0.1      40264      0x0011      372      85
10     2026-07-28T08:23:06.542821088-0400      127.0.0.1      40264      127.0.0.1      80      0x0010      85      373

(kali@kali)~/CIP-B103-Lab1
$
```

Screenshot 17: *tcp.stream eq 0* field extraction showing the full 10-frame TCP conversation, including the three-way handshake.

11. Part D - Examine the HTTP Request and Response

5. Filter for http.request and identify the GET method, request URI, Host and User-Agent.
6. Filter for http.response and record the status code, response phrase, server and content type.
7. Use Follow TCP Stream to reconstruct the complete request and response.
8. Compare the TCP payload length with the sequence/acknowledgement progression.

```
tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
```

```
tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
```



```
(kali@kali)-[~/CIP-B103-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.request' -T fields \
-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e http.request.method -e http.request.uri -e http.host -e http.user_agent \
| tee reports/http_requests.tsv
4      2026-07-28T08:23:06.532709748-0400      127.0.0.1      40264      127.0.0.1      80      GET      /basic.html      1
27.0.0.1      curl/8.20.0

(kali@kali)-[~/CIP-B103-Lab1]
$ tshark -r working/basic_working.pcapng -Y 'http.response' -T fields \
-e frame.number -e frame.time -e http.response.code -e http.response.phrase \
-e http.server -e http.content_type -e http.content_length \
| tee reports/http_responses.tsv
6      2026-07-28T08:23:06.541823214-0400      200      OK      Apache/2.4.68 (Debian)      text/html      120

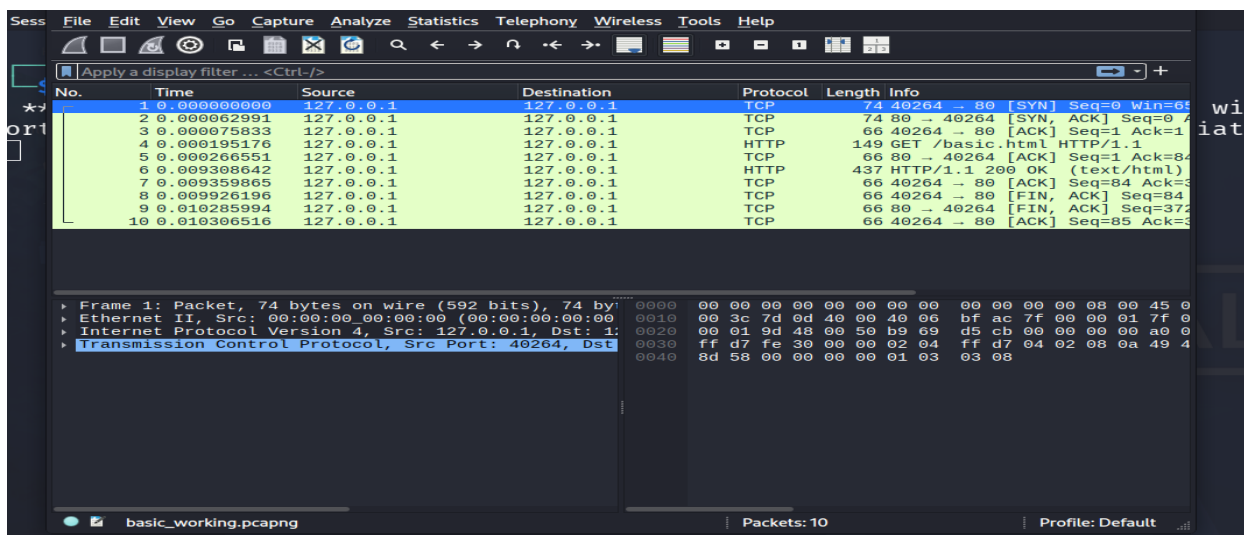
(kali@kali)-[~/CIP-B103-Lab1]
$
```

Screenshot 18: *http.request* and *http.response* field extraction showing the GET request and 200 OK response.

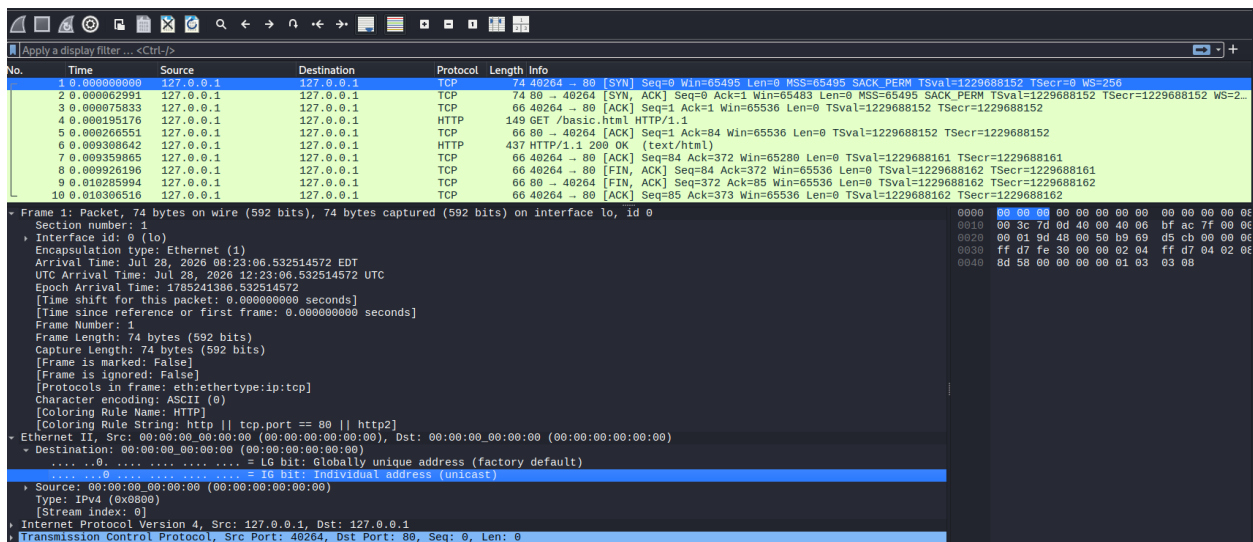
12. Part E - Inspect Encapsulation and Connection Closure

9. Select the HTTP GET frame and expand Frame, Internet Protocol, Transmission Control Protocol and Hypertext Transfer Protocol.
10. Explain the relationship among frame length, IP total length, TCP header length and TCP payload length.
11. Locate FIN/ACK packets or a reset that closes the session. Explain the observed termination sequence.
12. State why a loopback capture does not provide normal source/destination MAC addresses. For MAC evidence, repeat with two authorized VMs on the same host-only network.

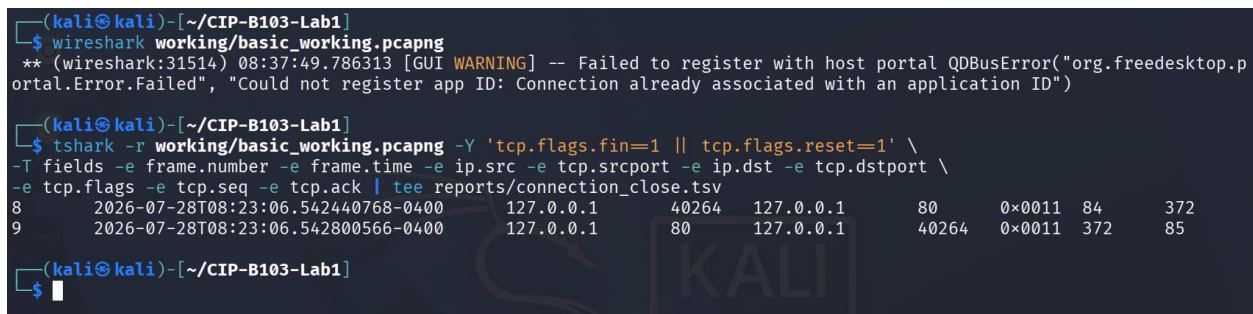
```
tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
-e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
```



Screenshot 19: *Wireshark* packet list showing the handshake, HTTP GET/response and connection teardown.



Screenshot 20: Expanded frame, Ethernet, IP and TCP layers for the GET request, showing loopback (all-zero) MAC addresses.



Screenshot 21: tshark filter for FIN/RST packets confirming the connection closure sequence.

13. Required Forensic Findings

Finding	Value/Observation
Capture start and end time	Started 08:23:06.532515 EDT (frame 1, SYN); ended 08:23:06.542821 EDT (frame 10, final ACK) — 28 Jul 2026, total span ~10.3 ms
Client IP and port	127.0.0.1 : 40264
Server IP and port	127.0.0.1 : 80
Client initial sequence number	0 (relative)
Server initial sequence number	0 (relative)
HTTP request timestamp	08:23:06.532710 EDT, 28 Jul 2026 (frame 4)

Finding	Value/Observation
Requested URI	/basic.html
HTTP response code	200 OK (frame 6)
Content type and length	text/html, 120 bytes
Connection termination method	Graceful FIN/ACK exchange: client FIN,ACK (frame 8, Seq=84 Ack=372), server FIN,ACK (frame 9, Seq=372 Ack=85), completed by client's final ACK (frame 10, Seq=85 Ack=373)
MAC-address observation	Ethernet source and destination both show 00:00:00_00:00:00 (all-zero placeholder) on the loopback interface; genuine MAC addresses are absent because lo traffic never traverses a real NIC — a two-VM host-only capture would be needed to observe real MACs

Required Screenshots Checklist

- ☐ Webpage showing your name and registration number.
- ☐ Apache service listening on TCP port 80.
- ☐ Capture running on the selected interface.
- ☐ Three-way handshake with sequence and acknowledgement numbers.
- ☐ HTTP GET request and complete headers.
- ☐ HTTP 200 response and HTML content.
- ☐ Follow TCP Stream output.
- ☐ TCP connection closure.
- ☐ Capture hashes.
- ☐ Optional two-VM capture showing source and destination MAC addresses.

Student Report Template

13. Cover page and executive summary.
14. Authority, scope and evidence description.

15. Environment and acquisition method.
16. Chronological packet timeline.
17. Handshake analysis.
18. HTTP request/response reconstruction.
19. Encapsulation and MAC-address explanation.
20. Findings, limitations and conclusion.
21. Appendix of commands, hashes and screenshots.

Submission Requirements

- One PDF report named CIP-B103-Lab1_RegNo_FullName.pdf.
- The original or generated PCAP/PCAPNG evidence file and its SHA-256 hash text file.
- A reports folder containing exported CSV/TXT command output and any recovered objects.
- Screenshots must be readable, numbered and referenced in the report narrative.
- Compress the report and evidence outputs into one ZIP named with your registration number and lab number.
- Do not submit passwords or decoded credentials in public discussion channels; include them only in the protected assessment submission where required.

Marking Rubric (100 Marks)

Assessment Area	Marks	What the Instructor Will Check
Preparation and evidence handling	10	Correct webpage, folders, capture preservation and hashes.
Traffic capture	15	Valid capture containing a complete HTTP session.
TCP handshake analysis	20	Correct flags, ports, timestamps, sequence and acknowledgement interpretation.
HTTP analysis	25	Correct request, response, headers, payload and stream reconstruction.

Assessment Area	Marks	What the Instructor Will Check
Encapsulation and closure	15	Correct layer and termination analysis, including loopback/MAC explanation.
Report quality	15	Clear timeline, screenshots, references and conclusions.

Instructor Quick Answer Guide

Checkpoint	Expected Observation
Default server port	TCP 80.
Handshake	SYN, SYN-ACK, ACK.
Request	GET /basic.html with Host and User-Agent headers.
Response	Usually HTTP/1.1 200 OK and text/html.
Loopback MACs	Normal Ethernet MAC addresses are not present on lo.
Sequence rule	ACK normally points to the next byte expected; SYN consumes one sequence number.

Command Reference Summary

Command	Purpose
curl -v URL	Generate an HTTP request and show headers.
tshark -i lo -w file.pcapng	Capture loopback traffic.
http.request	Wireshark display filter for HTTP requests.
tcp.stream eq 0	Display one TCP conversation.
Follow TCP Stream	Reconstruct application data.

Command	Purpose
sha256sum file	Create an integrity hash.

Academic Integrity and Authorization Statement

Student Declaration

I confirm that I completed this lab in an authorized environment, preserved the supplied evidence, did not target any third-party system, and accurately documented my own commands, observations and conclusions.